Vorlesung Rechnernetze
AIN 5

Laborübung

Paketanalyse mit Wireshark – Teil 1

(Einleitung und Teilaufgabe 2)

Prof. Dr. Dirk Staehle

Die Abgabe erfolgt durch Hochladen der bearbeiteten Word-Datei in Moodle.

Bearbeitung in Zweier-Teams

Team-Mitglied 1: Nicholas Plötz

Team-Mitglied 2: Robert Günther

1 Einleitung

Wireshark ist ein Werkzeug, mit dem Pakete, die über die Netzwerkschnittstelle eines Rechners laufen, aufgezeichnet und analysiert werden können. In dieser Übung lernen Sie Wireshark kennen und erarbeiten sich die Grundzüge der Datenübertragung im Internet auf den verschiedenen Protokollschichten. Die Übung besteht aus zwei Teilen. Nach diesem ersten Teil sollen Sie

- den Aufbau eines Pakets aus den Headern der verschiedenen Protokollschichten verstehen und erkennen, dass alle Pakete nach demselben Prinzip aufgebaut sind,
- die Header und Header-Felder eines Pakets in Wireshark finden und interpretieren können,
- HTTP-Verkehr im Klartext analysieren können und
- Verkehr gezielt mit Display-Filtern auswählen können.

Im zweiten Teil beschäftigen Sie sich mit verschlüsseltem Verkehr und mit der Analyse von Verkehrsflüssen.

1.1 Hinweise zur Bearbeitung

Wenn Sie Ihren eigenen Rechner nutzen, werden Ergebnisse unter Umständen nicht genau so aussehen oder funktionieren, wie es hier beschrieben ist, weil vieles von Betriebssystem und Browser abhängt. Zudem werden sich die Ergebnisse unterscheiden je nachdem, in welchem Netzwerk Sie sind: eduRoam, VPN oder zu Hause.

Das ist gewollt: Wenn Sie auf ein Problem oder eine unerwartete Beobachtung stoßen, versuchen Sie es zu erklären, diskutieren Sie es mit dem Betreuer im Labor oder nutzen Sie ein KI-Werkzeug. Halten Sie kurz fest, was das Problem war und wie Sie es gelöst haben. Solche Beobachtungen sind oft die interessantesten Antworten.

Fragen, die besonders zur Diskussion einladen, sind mit *[Diskussion]* markiert. Tragen Sie Ihre Antworten in die Felder unter den Fragen ein. Screenshots können Sie direkt in die Felder einfügen. Die Abgabe erfolgt eine Woche nach der Übung. Was Sie im Labor nicht schaffen, bearbeiten Sie bis dahin zu Hause.
Manche Aufgaben können Sie nicht basierend auf der bisherigen Vorlesung beantworten. nutzen Sie entweder Suchmaschinen oder KI, um die Fragen zu beantworten. Wichtig ist am Ende aber, dass Sie die Frage und die Antwort verstehen.

Mitschnitte speichern: Speichern Sie jeden Mitschnitt über Datei → Speichern unter im Format .pcapng. Sie benötigen die Mitschnitte teilweise in späteren Abschnitten und zum Weiterarbeiten zu Hause wieder. Verwenden Sie aussagekräftige Dateinamen, z. B. `abschnitt3_http.pcapng`. Die Mitschnitte bleiben auf Ihrem Rechner und werden nicht hochgeladen.

1.2 Datenschutz

Ein Mitschnitt kann persönliche Daten enthalten (z. B. Nachrichten anderer Anwendungen, Cookies). Schließen Sie vor dem Mitschneiden Anwendungen, die Sie nicht benötigen. Laden Sie keine Mitschnittdateien (.pcapng) hoch, sondern nur Ihre Antworten und Screenshots. Schneiden Sie ausschließlich Ihren eigenen Verkehr mit.

1.3 Vorbereitung vor der Übung

Erledigen Sie die folgenden Schritte vor der Übung, damit Sie im Labor direkt beginnen können:

- Installieren Sie Wireshark von https://www.wireshark.org/download.html. Unter Windows wird zusätzlich der Treiber Npcap installiert. Wählen Sie dabei die Option für den Loopback-Adapter („Support loopback traffic“) aus. Unter macOS installieren Sie das mitgelieferte Paket ChmodBPF. Unter Linux benötigt Ihr Benutzer Rechte zum Mitschneiden, siehe z. B. die Anleitungen weiter unten.
- Testen Sie, dass ein Mitschnitt funktioniert: Nach dem Start füllt sich die Paketliste.
- Prüfen Sie, dass im Terminal bzw. in der Eingabeaufforderung der Befehl `curl` verfügbar ist (unter Windows 10/11, macOS und den meisten Linux-Distributionen vorinstalliert).
- Für Variante C in Abschnitt 3 benötigen Sie Python 3. Unter Windows ist Python nicht vorinstalliert, Sie erhalten es unter https://www.python.org/downloads/.

Anleitungen für Linux:

- https://wiki.wireshark.org/CaptureSetup/CapturePrivileges
- https://wiki.archlinux.org/title/Wireshark
- https://itsfoss.com/install-wireshark-ubuntu/

2 Aufbau eines Pakets

Pakete werden von mehreren Protokollschichten verarbeitet: Jede Schicht betrachtet das Paket der darüberliegenden Schicht als Nutzdaten (Payload) und fügt vorne ihren eigenen Header an (Kapselung, siehe Vorlesung Kap. 2.2). Wireshark greift Pakete an der Netzwerkschnittstelle ab und sieht daher die Header aller Schichten. Im mittleren Bereich (Paketdetails) zeigt es diese als aufklappbare Zeilen an. Wenn Sie eine Zeile oder ein einzelnes Feld anklicken, markiert Wireshark die zugehörigen Bytes im Hex-Bereich darunter.

Starten Sie Wireshark und wählen Sie die Schnittstelle, über die Ihr Rechner mit dem Netz verbunden ist (meist WLAN[1]). Schneiden Sie 30 bis 60 Sekunden lang mit. Rufen Sie währenddessen einige Webseiten auf und führen Sie im Terminal `ping 8.8.8.8` aus.

1. Wählen Sie ein beliebiges Paket aus. Welche Zeilen zeigt Wireshark in den Paketdetails an? Ordnen Sie jede Zeile einer Schicht des Schichtenmodells aus der Vorlesung zu. Fügen Sie einen Screenshot ein.

[1] Wenn Sie Wireshark starten, sehen Sie bei der Auswahl der Interfaces, dass sich rechts neben einigen Interfaces eine Kurve aufbaut. Über diese Interfaces wird Verkehr übertragen. In Windows können Sie mittels „ipconfig /all“ (Ausruf in der Konsole) feststellen, welcher IP Adresse der jeweilige Inerfacename zugeordnet ist. Unter Linux ist das „ip addr“ oder „ifconfig“, unter macOS „ifconfig“.

```
▸ Frame 26087: Packet, 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface wlo1, id 0
▸ Ethernet II, Src: AzureWaveTec_97:99:74 (f0:68:e3:97:99:74), Dst: 7e:55:6d:fe:3c:f9 (7e:55:6d:fe:3c:f9)
▸ Internet Protocol Version 4, Src: 10.48.182.223, Dst: 104.29.154.102
▸ User Datagram Protocol, Src Port: 59442, Dst Port: 19324
▸ Real-time Transport Control Protocol (Generic RTP Feedback)
▸ [Malformed Packet: RTCP]
```

Network Access Layer:

Ethernet

```
▾ Ethernet II, Src: AzureWaveTec_97:99:74 (f0:68:e3:97:99:74), Dst: 7e:55:6d:fe:3c:f9 (7e:55:6d:fe:3c:f9)
  ▾ Destination: 7e:55:6d:fe:3c:f9 (7e:55:6d:fe:3c:f9)
      .... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  ▾ Source: AzureWaveTec_97:99:74 (f0:68:e3:97:99:74)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
    [Stream index: 0]
▾ Internet Protocol Version 4, Src: 10.48.182.223, Dst: 104.29.154.102
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 72
    Identification: 0x5593 (21907)
  ▸ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: UDP (17)
    Header Checksum: 0x217f [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.48.182.223
```

Transport Layer: IPV4

```
▾ Internet Protocol Version 4, Src: 10.48.182.223, Dst: 104.29.154.102
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 72
    Identification: 0x5593 (21907)
  ▸ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: UDP (17)
    Header Checksum: 0x217f [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.48.182.223
    Destination Address: 104.29.154.102
    [Stream index: 1]
```

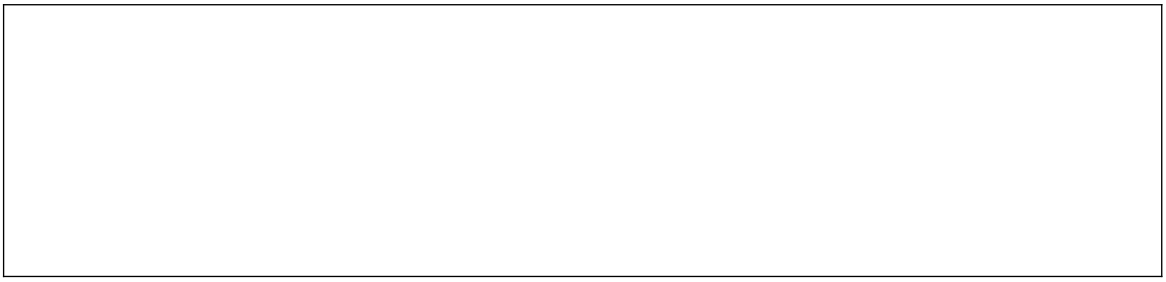
Application layer: RTCP

```
▾ Real-time Transport Control Protocol (Generic RTP Feedback)
    10.. .... = Version: RFC 1889 Version (2)
    ..0. .... = Padding: False
    ...0 1111 = RTCP Feedback message type (FMT): Transport-wide Congestion Control (Transport-cc) (15)
    Packet type: Generic RTP Feedback (205)
    Length: 5 (24 bytes)
    Sender SSRC: 0x000031e4 (12772)
    Media source SSRC: 0xb7ae8d16 (3081669910)
  ▸ Transport-cc
```

Vivaldi

Transport layer:UDP

```
▾ User Datagram Protocol, Src Port: 59442, Dst Port: 19324
    Source Port: 59442
    Destination Port: 19324
    Length: 52
    Checksum: 0x1120 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 3]
    [Stream Packet Number: 7151]
  ▸ [Timestamps]
    UDP payload (44 bytes)
```



2. Klappen Sie nacheinander den Ethernet-, den IP- und den TCP- bzw. UDP-Header auf. Welche Felder enthält jeder Header, wie viele Bytes ist er lang, und welche Werte haben die Felder in Ihrem Paket? Klicken Sie die Felder an und beobachten Sie, welche Bytes im Hex-Bereich markiert werden.

```
▾ Ethernet II, Src: AzureWaveTec_97:99:74 (f0:68:e3:97:99:74), Dst: 7e:55:6d:fe:3c:f9 (7e:55:6d:fe:3c:f9)
  ▾ Destination: 7e:55:6d:fe:3c:f9 (7e:55:6d:fe:3c:f9)
      .... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  ▾ Source: AzureWaveTec_97:99:74 (f0:68:e3:97:99:74)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
    [Stream index: 0]
```

Source, Destination und Typ.
Länge: 14 bytes
Werte:

```
7e 55 6d fe 3c f9 f0 68  e3 97 99 74 08 00 45 00   ~Um·<··h ···t··E·
00 48 55 93 40 00 40 11  21 7f 0a 30 b6 df 68 1d   ·HU·@·@· !··0··h·
9a 66 e8 32 4b 7c 00 34  11 20 8f cd 00 05 00 00   ·f·2K|·4 · ·····
31 e4 b7 ae 8d 16 27 3b  5a c1 dd da ff 1a 74 c0   1·····'; Z·····t·
c7 30 61 78 b1 fb e3 cd  8e d9 b9 ee fa 5f df 6d   ·0ax···· ·····_·m
b2 5e 89 0d 00 80                                 ·^····
```

```
▾ Internet Protocol Version 4, Src: 10.48.182.223, Dst: 104.29.154.102
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 72
    Identification: 0x5593 (21907)
  ▸ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: UDP (17)
    Header Checksum: 0x217f [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.48.182.223
    Destination Address: 104.29.154.102
    [Stream index: 1]
```

Source, Destination, genutzte Protokolle und Identifikation
Länge: 20 bytes.
Werte:

```
7e 55 6d fe 3c f9 f0 68  e3 97 99 74 08 00 45 00   ~Um·<··h ···t··E·
00 48 55 93 40 00 40 11  21 7f 0a 30 b6 df 68 1d   ·HU·@·@· !··0··h·
9a 66 e8 32 4b 7c 00 34  11 20 8f cd 00 05 00 00   ·f·2K|·4 · ·····
31 e4 b7 ae 8d 16 27 3b  5a c1 dd da ff 1a 74 c0   1·····'; Z·····t·
c7 30 61 78 b1 fb e3 cd  8e d9 b9 ee fa 5f df 6d   ·0ax···· ·····_·m
b2 5e 89 0d 00 80                                 ·^····
```

```
▾ User Datagram Protocol, Src Port: 59442, Dst Port: 19324
    Source Port: 59442
    Destination Port: 19324
    Length: 52
    Checksum: 0x1120 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 3]
    [Stream Packet Number: 7151]
  ▸ [Timestamps]
    UDP payload (44 bytes)
```

Sourceport, Destport Länge

Länge: 8 byte(44 byte payload)

Werte:

```
7e 55 6d fe 3c f9 f0 68  e3 97 99 74 08 00 45 00   ~Um·<··h ···t··E·
00 48 55 93 40 00 40 11  21 7f 0a 30 b6 df 68 1d   ·HU·@·@· !··0··h·
9a 66 e8 32 4b 7c 00 34  11 20 8f cd 00 05 00 00   ·f·2K|·4 · ······
31 e4 b7 ae 8d 16 27 3b  5a c1 dd da ff 1a 74 c0   1·····'; Z·····t·
c7 30 61 78 b1 fb e3 cd  8e d9 b9 ee fa 5f df 6d   ·0ax···· ·····_·m
b2 5e 89 0d 00 80                                 ·^····
```

```
▾ Real-time Transport Control Protocol (Generic RTP Feedback)
    10.. .... = Version: RFC 1889 Version (2)
    ..0. .... = Padding: False
    ...0 1111 = RTCP Feedback message type (FMT): Transport-wide Congestion Control (Transport-cc) (15)
    Packet type: Generic RTP Feedback (205)
    Length: 5 (24 bytes)
    Sender SSRC: 0x000031e4 (12772)
    Media source SSRC: 0xb7ae8d16 (3081669910)
  ▸ Transport-cc
```

Version, Messsage type, Length

Länge: 24 bytes

Werte:

```
7e 55 6d fe 3c f9 f0 68  e3 97 99 74 08 00 45 00   ~Um·<··h ···t··E·
00 48 55 93 40 00 40 11  21 7f 0a 30 b6 df 68 1d   ·HU·@·@· !··0··h·
9a 66 e8 32 4b 7c 00 34  11 20 8f cd 00 05 00 00   ·f·2K|·4 · ······
31 e4 b7 ae 8d 16 27 3b  5a c1 dd da ff 1a 74 c0   1·····'; Z·····t·
c7 30 61 78 b1 fb e3 cd  8e d9 b9 ee fa 5f df 6d   ·0ax···· ·····_·m
b2 5e 89 0d 00 80                                 ·^····
```

3. Suchen Sie sich mithilfe von Display-Filtern vier unterschiedliche Pakete: ein TCP-Paket (`tcp`), eine DNS-Anfrage (`dns`), ein Paket Ihres Ping (`icmp`) und ein ARP-Paket (`arp`). Wie sind die vier Pakete aufgebaut? Welche Header haben alle gemeinsam, und worin unterscheiden sie sich? *[Diskussion]* Falls Sie kein ARP-Paket finden: Woran könnte das liegen?

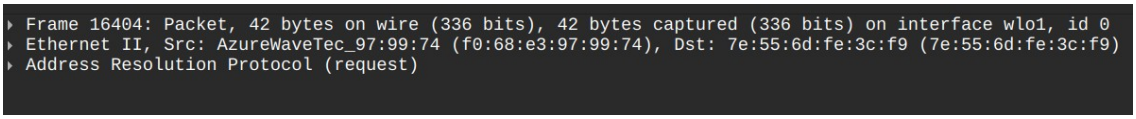
tcp

TCP



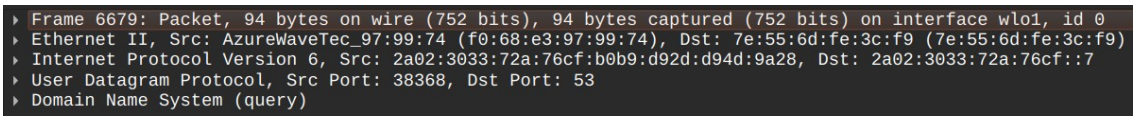
ARP

dns



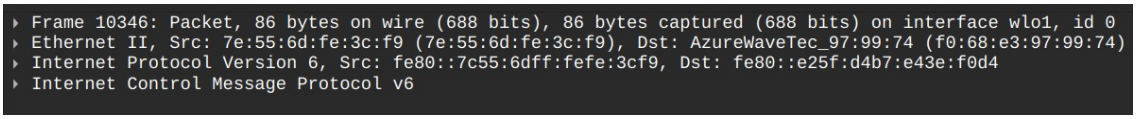
DNS

ARP



ICMPv6

ICMPv6



(nur notwendiges ICMPv6 gefunden)

Ethernet: alle
IPV6: alle außer ARP
ansonsten werden spezifische botschaften übermittelt

4. Jeder Header enthält ein Feld, das angibt, welcher Header als Nächstes folgt. Finden Sie dieses Feld im Ethernet-Header und im IP-Header und notieren Sie seine Werte für Ihre vier Pakete. Wie entscheidet der Empfänger beim TCP- und beim DNS-Paket, an welche Anwendung die Nutzdaten weitergegeben werden?

TCP:
ethernet = 0x0800
IP = TCP(6)

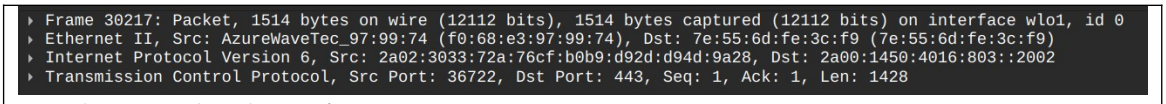
IP-Adressen
ICMPv6:
ethernet: 0x86dd
IP: ICMPv6 (58)
ARP:

DNS:
ethernet && IP: UDP (17)
über die IP Adressen und die Query

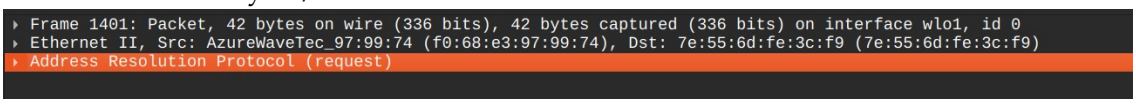
5. Warum hat das ARP-Paket keinen IP-Header?

Arbeitet auf sicherungsübereignet des Osi Modells nicht auf der Vermittlungsschicht

6. Suchen Sie das kleinste und das größte Paket in Ihrem Mitschnitt (Sortierung nach der Spalte „Length“). Welchen Anteil haben die Header jeweils an der Paketgröße?



Header Anteil 86 bytes/1514.-›5.68%



Header Anteil 42/42-›100%

Das folgende HTTP-Paket wurde vor einigen Jahren mit Wireshark aufgezeichnet. Es ist so dargestellt wie im Hex-Bereich von Wireshark: links der Offset, in der Mitte die Bytes als Hexadezimalzahlen, rechts dieselben Bytes als ASCII-Zeichen.

Ethernet
IP
TCP
HTTP-Nutzdaten

```
0000  38 22 d6 67 19 00 00 21 cc 63 82 2c 08 00 45 00   8".g...!.c.,..E.
0010  02 9c 02 ed 40 00 80 06 40 66 8d 25 1d 5d 5b c6   ....@...@f.%.][.
0020  ae c0 e2 26 00 50 4f 4c 29 24 72 ce 3c d4 50 18   ...&.POL)$r.<.P.
0030  40 b0 62 e7 00 00 47 45 54 20 2f 77 69 6b 69 2f   @.b...GET /wiki/
0040  53 69 6d 70 6c 65 5f 53 65 72 76 69 63 65 5f 44   Simple_Service_D
0050  69 73 63 6f 76 65 72 79 5f 50 72 6f 74 6f 63 6f   iscovery_Protoco
0060  6c 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73 74   l HTTP/1.1..Host
0070  3a 20 64 65 2e 77 69 6b 69 70 65 64 69 61 2e 6f   : de.wikipedia.o
0080  72 67 0d 0a 55 73 65 72 2d 41 67 65 6e 74 3a 20   rg..User-Agent:
0090  4d 6f 7a 69 6c 6c 61 2f 35 2e 30 20 28 57 69 6e   Mozilla/5.0 (Win
00a0  64 6f 77 73 20 4e 54 20 36 2e 31 3b 20 57 4f 57   dows NT 6.1; WOW
00b0  36 34 3b 20 72 76 3a 33 32 2e 30 29 20 47 65 63   64; rv:32.0) Gec
```

7. Markieren Sie im Hex-Dump den Ethernet-Header, den IP-Header, den TCP-Header und die HTTP-Nutzdaten in unterschiedlichen Farben (z. B. mit der Texthervorhebung von Word). Nutzen Sie dafür Ihre Erkenntnisse aus den Fragen 2 bis 4.

8. Was sind Quell- und Ziel-MAC-Adresse, Quell- und Ziel-IP-Adresse sowie Quell- und Ziel-Port? Welchen Wert haben die Felder, die angeben, welcher Header folgt? Geben Sie die IP und MAC Adressen in der üblichen Schreibweise an.

MAC-Dest = `38:22:d6:67:19:00`
`MAC-Source = 00:21:cc:63:82:2c`
IPV4-Source = `8d 25 1d 5d = 141.37.29.93`
IPV4-Dest = `5b c6 ae c0 = 91.198.174.192`
Port-Source = e2 26
Port-Dest = 00 50
Next Header-Eth = 0x800
IP-Next header = 06

9. *[Diskussion]* Warum könnten Sie ein Paket wie dieses beim Aufruf von Wikipedia heute nicht mehr so mitschneiden?

Heutztage wird IPv6 genutzt nicht IPv4.